MiroFish: simulate objections, then validate with real customers

Researched 2026-11-02.

What this gives you

MiroFish is a multi-agent social simulation tool, not a panel of paying customers. The reel's startup test is a useful hypothesis-generation example, but generated enthusiasm is not evidence of purchases, retention or market size. The inspected sources do not establish that its reports reliably predict startup outcomes. Initial reel evidence came from the Muse dossier; this guide adds an independent check on November 2, 2026.

The current repository (https://github.com/666ghj/MiroFish) describes seed documents, graph construction, personas, social interactions and a generated report. Its simulation engine uses OASIS (https://github.com/camel-ai/oasis). That supports trying alternative narratives and exploring objections; it does not turn simulated posts into observed customer behavior.

Install and run

Prerequisites from the upstream README: Node.js 18+, Python 3.11-3.12 and uv. These are documented commands, not a claim that Kiln installed or benchmarked the software. In a new folder, use:

```
git clone https://github.com/666ghj/MiroFish.git
cd MiroFish
```

On Windows PowerShell, copy the example configuration:

```
Copy-Item .env.example .env
```

Edit `.env` locally. The exact names are `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL_NAME` and `ZEP_API_KEY`; the dossier's shorter `LLM_MODEL` is not the documented variable. Use a real supported model and endpoint. Remove the optional `LLM_BOOST_*` block if unused rather than retaining example credentials. See the actual template (https://github.com/666ghj/MiroFish/blob/main/.env.example).

```
npm run setup:all
npm run dev
```

The documented UI is localhost:3000 and backend localhost:5001. Docker users can instead run `docker compose up -d` after configuring the same `.env`. The documented default uses a hosted LLM endpoint and Zep Cloud. Self-hosting the UI does not establish that all submitted information stays on the machine. The project lists AGPL-3.0 licensing; inspect the repository license when evaluating redistribution.

Run a useful experiment

1. Write one specific customer segment, problem, proposed price and purchase action. Remove confidential or personal records from the seed material.
2. Produce two briefs differing in one factor, such as price or onboarding effort. Keep model, persona mix, rounds and seed information recorded.
3. Begin with a small run and a spending limit. The upstream README warns about high consumption and suggests fewer than 40 rounds initially.
4. Extract objections and contradictory reactions, not just positive quotes. Repeat with changed assumptions to see which conclusions are fragile.
5. Turn the strongest uncertainty into real interviews, a landing-page test or a small paid pilot. Record which simulated predictions failed.

Reusable prompt

```
Review this simulated startup report as a source of hypotheses. Separate
observed input facts from generated agent behavior. List assumptions that
could reverse the conclusion, give counterexamples, and propose three real
customer tests with explicit pass/fail criteria. Do not infer market size,
purchase probability or revenue from simulated likes or comments. Cite the
seed evidence for factual statements and label missing evidence plainly.
```

Project idea and next step

Build a simulation-to-validation ledger: scenario, model, persona assumptions, generated objection, real test, observed result and decision. A blank CSV accompanies this guide. Start by filling one scenario; leave outcome cells empty until measured. The valuable deliverable is an auditable set of hypotheses and real follow-up evidence, not a synthetic approval score.

Original sources

- https://www.instagram.com/reel/DfNWVOTviIP/